

The Vision Library

Ball Balancing Platform — student guide

Ball Balancing Platform project

Library version 1.0 — October 6, 2026

Abstract

This guide covers everything you can use in `tracking.cpp`: the camera image you receive, what the ball looks like in it and why, the conventions for reporting where you found the ball, and how fast your code has to be to keep up on the FireBeetle. It then walks through the starter tracker and the standard techniques for making it more accurate and more robust [2, 4].

Contents

1	Your job	1
2	The image	1
3	What the camera sees	2
3.1	Image and plate directions	2
3.2	Scale and distortion	3
3.3	Imperfections you will meet	3
4	The starter tracker	3
5	Techniques	4
6	Speed: the FireBeetle budget	4
7	Checking your tracker	5
8	Things to try	5

1 Your job

```
void track(const Image& image, Detection& ball);
```

The library calls `track()` once for every camera frame, 30 times a second by default. Find the ball and report its centre **in pixels**:

```
ball.found = true;
ball.x = 163.4f; // column
ball.y = 118.9f; // row
```

If you cannot find it, set `ball.found = false`. Do not guess. The library then converts your pixel position to millimetres on the plate, accounting for the lens and for refraction through the plate, and hands it to `control()`. You never deal with millimetres here.

2 The image

<code>image.width(), image.height()</code>	320 × 240 pixels (QVGA).
<code>image.gray(x, y)</code>	Brightness 0–255.
<code>image.red(x, y), green, blue</code>	Colour channels, 0–255 each.

- Pixel $(0,0)$ is the **top-left** corner; x grows to the right and y grows down. Pixel (i,j) covers the square $[i, i+1) \times [j, j+1)$, so its centre is at $(i+0.5, j+0.5)$. Report the ball centre in the same continuous coordinates, which is why the starter code adds 0.5.
- The camera delivers **RGB565**: 5 bits of red, 6 of green and 5 of blue. Each channel is scaled back to 0–255, but only has 32 (red, blue) or 64 (green) distinct levels. Smooth gradients therefore show bands (fig. 1, right).
- `gray()` is the standard luma, computed in integers: $Y = (77R + 150G + 29B + 128)/256$ (ITU-R BT.601 weights).
- Reading outside the image is undefined and can crash your program. Keep $0 \leq x < \text{width}()$ and $0 \leq y < \text{height}()$.

3 What the camera sees

The camera sits 134 mm below the plate’s top surface, at its centre, looking *up* through the transparent acrylic plate. So in the image:

- The **ceiling** is the bright background. It is slightly brighter on one side (a window), and darker towards the corners because of lens vignetting.
- The **plate** is a large disc, very slightly darker than its surroundings ($\approx 8\%$ of the light is reflected at its surfaces).
- The three **joint bosses** under the plate are small dark blobs, about 100 px from the centre, at the leg positions. They are the classic false detection.
- The **ball** is seen from below. A steel ball looks dark, because it reflects the dark base, with a brighter rim. With the default lighting a white ball’s underside is also dark; switch on the ring light (Simulated hardware) to light it up. A 20 mm ball is about 22 px across.

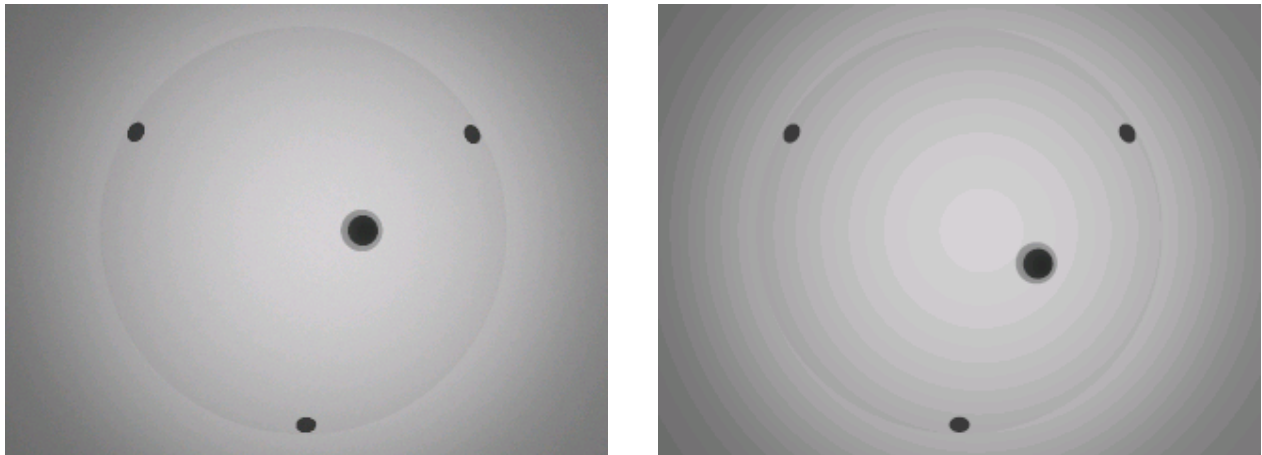


Figure 1: Simulated camera frames. Left: a normal frame, with noise. Right: noise switched off (Simulated hardware, image noise = 0), which shows the RGB565 banding of the vignetted ceiling.

3.1 Image and plate directions

With the plate level, the image centre $(160, 120)$ looks at the plate centre. Moving the ball towards plate $+x$ moves it **right** in the image, and towards plate $+y$ moves it **down** (fig. 2). Because the camera looks from below, the image is a mirror image of what you see standing over the platform. You only report pixels, so this does not affect your code. It does matter when you compare the camera view with the 3D view.

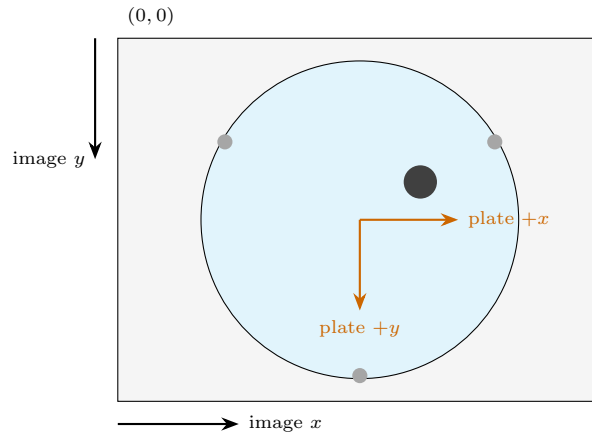


Figure 2: The camera image with the plate's axes overlaid. Plate $+y$ points *down* in the image, because the camera looks up from below.

3.2 Scale and distortion

At the centre of the image one pixel is about 0.9 mm at the height of the ball's centre. The wide-angle lens has barrel distortion, which squeezes the edges: there, one pixel covers more millimetres. The library corrects for this, and for the ≈ 1 mm sideways shift that the 5 mm acrylic plate causes near its edge. Your job is only to find the right pixel.

3.3 Imperfections you will meet

- **Noise:** every pixel fluctuates by a few grey levels from frame to frame (photon shot noise and read noise).
- **Motion blur:** the exposure lasts 6 ms. A ball moving at 200 mm/s moves 1.2 mm, more than a pixel, during it.
- **Rolling shutter:** the rows are captured one after another over 15 ms, so a fast ball is slightly skewed.
- **Perspective:** the camera sees a sphere's silhouette, whose centre is not exactly the projection of the ball's centre. Near the plate edge the difference is about 1 mm.

You can change the noise, the exposure, the lens and the lighting in the simulator to see how your tracker copes.

4 The starter tracker

```
void track(const Image& image, Detection& ball) {
    const int threshold = 80;           // pixels darker than this belong to the ball
    const float searchRadius = 90;      // ignore the dark joint bosses near the plate edge
    const float cx = image.width() / 2.0f, cy = image.height() / 2.0f;
    long sumX = 0, sumY = 0, count = 0;
    for (int y = 0; y < image.height(); y++)
        for (int x = 0; x < image.width(); x++) {
            const float dx = x - cx, dy = y - cy;
            if (dx * dx + dy * dy > searchRadius * searchRadius) continue;
            if (image.gray(x, y) < threshold) { sumX += x; sumY += y; count++; }
        }
    if (count < 20) { ball.found = false; return; }
    ball.found = true;
    ball.x = float(sumX) / count + 0.5f;
    ball.y = float(sumY) / count + 0.5f;
}
```

It *thresholds* the image (dark pixels are ball) and takes the *centroid* of the selected pixels:

$$\bar{x} = \frac{1}{N} \sum_{(x,y) \in S} x + \frac{1}{2}, \quad \bar{y} = \frac{1}{N} \sum_{(x,y) \in S} y + \frac{1}{2}. \quad (1)$$

It works, but it has weaknesses you can fix:

1. The circular mask that hides the bosses also hides the ball when it is near the plate edge.
2. The fixed threshold breaks when the lighting, exposure or ball changes.
3. Every pixel is visited every frame, which is slow on the FireBeetle.
4. A binary centroid ignores partly covered edge pixels, so it is less precise than it could be.

5 Techniques

Weighted centroid. Weigh each pixel by how much darker it is than the background, $w = \max(0, B - Y)$, and compute $\bar{x} = \sum wx / \sum w$. Edge pixels then contribute in proportion to how much of the ball they contain, which gives sub-pixel accuracy.

Adaptive threshold. Choose the threshold from the image itself. One way is the midpoint between the typical background and ball brightness. Another is Otsu's method, which picks the threshold that best separates the histogram into two classes [3].

Search window (tracking). The ball moves at most a few pixels between frames. Search only a window, say 50×50 pixels, around the last position, and fall back to the whole image when you lose it. This is about $30\times$ less work, and the bosses are excluded automatically unless the ball is right next to one.

Connected components. Group neighbouring dark pixels into blobs and keep the one that looks like the ball: the right area (≈ 380 px for 20 mm), round, and close to where it was. This rejects bosses and other dark specks wherever the ball is [2].

Background subtraction. Store an image of the empty plate (or a slowly updated average), then look for pixels that differ from it. This works for any ball colour and removes the vignetting and the window gradient.

Circle fitting. Find edge pixels and fit a circle to them, or vote for circle centres with a Hough transform [1]. This is more robust when the ball is partly occluded, but much more expensive.

Sanity checks. Reject a detection whose area is far from the expected size, or that jumped implausibly far, and report `found = false` instead. A missing measurement is better than a wrong one: `control()` sees `ball.found` and can hold still.

6 Speed: the FireBeetle budget

On the real platform your code runs on the FireBeetle's ESP32-S3 at 240 MHz, many times slower than a PC. The simulator measures how long your `track()` and `control()` take on the PC, multiplies by the slowdown factor in Setup, and delays the result by that much. The Simulation tab shows both numbers.

- At 30 fps there are 33 ms per frame. Time spent in your code is delay in the control loop, so less is better (see the control library guide).
- A full scan visits 76 800 pixels. Each `gray()` call converts RGB565 to luma. Scan a window, or every second pixel for a coarse search, instead.
- Use `float` and `int`, never `double`. The ESP32-S3 has no hardware for `double`.
- Avoid allocating memory (`new`, `std::vector` growth) inside `track()`.

7 Checking your tracker

The simulator knows exactly where the ball is. In the Simulation tab:

- The camera view draws your detection as a green cross on the frame your code received.
- The *ball x* and *ball y* plots show the true position (line) and your measurement (dots). The gap between them is your tracking error, after the library's conversion to millimetres.
- `log("name", value)` plots anything else, for example your blob area or the threshold you chose.

8 Things to try

1. Replace the binary centroid with a weighted centroid. Compare the measurement noise in the plots with the image noise set to 2.
2. Add a search window around the last position, and compare the FireBeetle time shown in the Simulation tab.
3. Make the tracker work for the white ball with the ring light on, without changing the threshold by hand.
4. Remove the circular mask and reject the bosses with a blob-size check instead. Can you now track the ball all the way to the plate edge?

References

- [1] Richard O. Duda and Peter E. Hart. Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1):11–15, 1972.
- [2] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4th edition, 2018.
- [3] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979.
- [4] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2nd edition, 2022.